

LAPORAN PRAKTIKUM STRUKTUR DATA

“ALGORITMA SORTING LANJUTAN”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadiran Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-8 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus kepada pembahasan tiga algoritma sorting lanjutan dari praktikum sebelumnya. Praktikum ini membahas algoritma Merge Sort, Quick Sort, Shell Sort dan tambahan untuk program GUI yang akan menjadi pengaplikasian dari algoritma Bubble Sort.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami konsep dan mengidentifikasi perbedaan dari jenis jenis algoritma sorting. Melalui projek sederhana, penulis dapat memahami alur logika dari masing masing algoritma sorting.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhilila

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Program ShellSort_2511531001	3
2.2.1 Kode Program	3
2.2.2 Penjelasan.....	3
2.2.3 Output.....	4
2.3 Program QuickSort_2511531001	4
2.3.1 Kode Program	4
2.3.2 Penjelasan.....	6
2.3.3 Output.....	7
2.4 Program MergeSort_2511531001	7
2.4.1 Kode Program	7
2.4.2 Penjelasan.....	9
2.4.3 Output.....	9
2.5 Program BubbleSortGUI_2511531001	9
2.5.1 Kode Program	9
2.5.2 Penjelasan.....	14
2.5.3 Output.....	16
BAB III KESIMPULAN.....	18
DAFTAR PUSTAKA	19
LAPORAN TUGAS.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, kita sudah membahas tiga algoritma Sorting dengan langsung mengimplementasikannya ke dalam program sederhana. Dari pengimplementasian ini, kita dapat mengetahui berapa lama waktu yang dibutuhkan untuk mengeksekusi program pengurut yang menggunakan masing masing dari algoritma yang sudah dipelajari. Dalam praktikum kali ini, kita akan lebih mengenal tiga algoritma pengurutan atau sorting lainnya. Sehingga output akhir dari praktikum ini adalah untuk mengetahui enam algoritma sorting yang berbeda beserta mengetahui kompleksitas waktu pada masing masing algoritma.

Adapun tiga algoritma yang akan dibahas dalam praktikum ini adalah algoritma Quick Sort, Merge Sort, dan Shell Sort. Setelah itu, akan ada satu program GUI yang akan mengimplementasikan salah satu algoritma yang juga sudah dipelajari pada praktikum selanjutnya yaitu adalah algoritma Bubble Sort.

1.2 Tujuan

1. Memahami konsep dasar dari Algoritma Sorting.
2. Mengidentifikasi perbedaan dalam penggunaan jenis jenis Algoritma Sorting.
3. Mengimplementasikan algoritma Sorting pada program yang menggunakan GUI.

1.3 Manfaat

1. Memperdalam pemahaman tentang prinsip dasar pengurutan data dan logika.
2. Melatih kemampuan menerjemahkan logika/pseudocode untuk mengimplementasikan kode yang benar, rapi juga efisien.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Shell Sort

Shell Sort adalah algoritma sorting berbasis perbandingan di tempat. Algoritma ini menggunakan prinsip untuk membandingkan suatu elemen dengan elemen lain dengan jarak tertentu sehingga membentuk sebuah sub-list. Penukaran akan dilakukan jika diperlukan. Jarak yang digunakan dalam algoritma ini disebut gap. Proses Shell Sort secara umum adalah dengan cara membuat sub-list yang didasarkan pada gap yang digunakan. Lalu, urutkan masing masing sub-list dengan cara menukar data yang membutuhkan penukaran. Setelah itu baru gabungkan seluruh sub-list yang ada.

2.1.2 Merge Sort

Merge Sort adalah algoritma yang memiliki prinsip divide dan conquer. Algoritma ini akan memecah data terlebih dahulu dan menyelesaikan masing masing bagian data sebelum menggabungkan keseluruhan data. Cara kerja umum algoritma ini adalah membagi dua keseluruhan data terlebih dahulu, dan hasil pembagian pertama ini akan kembali dibagi menjadi dua dan pembagian ini akan dilakukan secara berulang sampai masing masing bagian hanya memiliki satu data di dalamnya.

2.1.3 Quick Sort

Algoritma Quick Sort adalah algoritma yang akan mengurutkan data dengan cara membagi data tersebut menjadi partisi partisi dan menentukan angka pivot. Data kemudian akan dipecah menjadi dua bagian yaitu data yang kurang atau lebih kecil dari pivot dan data yang lebih besar dari pivot. Proses ini akan terjadi secara rekursif hingga semua partisi partisi data terurut.

2.2 Program ShellSort_2511531001

2.2.1 Kode Program

```
package pekan8_25511531001;
public class ShellSort_2511531001 {
    public static void ShellSort_2511531001(int [] A) {
        int n_1001 = A.length;
        int gap_1001 = n_1001/2;
        while (gap_1001 >0) {
            for ( int i_1001 = gap_1001 ; i_1001 <
n_1001; i_1001++) {
                int temp_1001 = A[i_1001];
                int j_1001 = i_1001;
                while ( j_1001>= gap_1001 &&
A[j_1001 - gap_1001 ] > temp_1001) {
                    A[j_1001]= A[j_1001 -
gap_1001];
                    j_1001 = j_1001 -
gap_1001;
                }
                A[j_1001] = temp_1001;
            }
            gap_1001 = gap_1001/2;
        }
    }

    public static void main (String [] args) {
        int [] data_1001 = {3, 10, 4, 6, 8, 9, 7, 2,
1, 5};

        System.out.print("Sebelum : ");
        printArray_2511531001(data_1001);

        ShellSort_2511531001(data_1001);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_2511531001(data_1001);
    }

    public static void printArray_2511531001 (int []
arr) {
        for (int i_1001 : arr)
            System.out.print(i_1001 + " ");
        System.out.println();
    }
}
```

Program 2.1 : Program Algoritma ShellSort

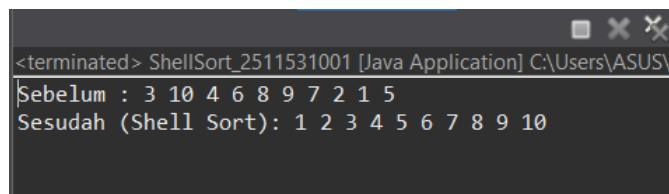
2.2.2 Penjelasan

Program ini dimulai dengan membuat method ShellSort dengan array A sebagai objeknya. Setelah itu, panjang dari array akan disimpan dalam variabel n dan nilai gap adalah hasil pembagian n dengan 2. Lalu, program akan menggunakan

perulangan while dimana syarat dari perulangan ini akan terus berjalan adalah nilai gap masih besar dari 0. Di dalam perulangan while sendiri, ada perulangan for. Perulangan ini dimulai dari elemen dengan indeks sebesar gap dan akan terus berulang hingga akhir dari array. Perulangan for juga diisi dengan perulangan while dimana selagi variabel j masih lebih besar atau sama dengan gap dan elemen yang berada di indeks $j - \text{gap}$ lebih besar daripada temp, maka elemen tersebut akan digeser ke kanan. Setelah itu, elemen dengan indeks j akan dijadikan temp.

Program akan memanggil method Shell Sort pada program main. Setelah mengisi array dengan elemen dan menampilkan array yang belum terurut, method shell sort akan dipanggil dan setelah itu program akan menampilkan array yang sudah terurut dengan method printArray.

2.2.3 Output



```
<terminated> ShellSort_2511531001 [Java Application] C:\Users\ASUS\
Sebelum : 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.1 : Output dari program algoritma Shell Sort

2.3 Program QuickSort_2511531001

2.3.1 Kode Program

```
package pekan8_2511531001;
public class QuickShort_2511531001 {
    static void swap_1001 (int [] arr_1001, int i_1001,
int j_1001) {
        int temp_1001 = arr_1001 [i_1001];
        arr_1001[i_1001]= arr_1001[j_1001];
        arr_1001[j_1001]= temp_1001;
    }

    //metode tambahan untuk mengatur pivot menggunakan
median of three
    static void medianOfThree_1001(int[] arr_1001, int
low_1001, int high_1001) {
        int mid_1001 = low_1001 + (high_1001 -
low_1001)/2;

        //urutlan elemen low, mid, high
```

```

        if (arr_1001[low_1001] > arr_1001[mid_1001])
        {
            swap_1001(arr_1001, low_1001,
high_1001);
        }

        if (arr_1001[low_1001] > arr_1001[high_1001])
        {
            swap_1001(arr_1001, low_1001,
high_1001);
        }

        if (arr_1001[low_1001] > arr_1001[high_1001])
        {
            swap_1001(arr_1001, mid_1001,
high_1001);
        }
        swap_1001 (arr_1001, mid_1001, high_1001);
    }

    static int partition_1001 (int[] arr_1001, int
low_1001, int high_1001) {
        //panggil fungsi medianOfThree sebelum
menentukan pivot
        medianOfThree_1001 (arr_1001, low_1001,
high_1001);

        int pivot_1001 = arr_1001 [high_1001];
        int i_1001 = (low_1001 -1);

        for (int j_1001 = low_1001; j_1001 <=
high_1001-1; j_1001++) {
            //jika elemen saat ini lebih kecil dari
atau sama dengan pivot
            if (arr_1001[j_1001] < pivot_1001) {
                //increment indeks elemen yang
lebih kecil
                i_1001++;
                swap_1001(arr_1001, i_1001,
j_1001);
            }
        }
        swap_1001(arr_1001,i_1001+1,high_1001);
        return (i_1001+1);
    }

    static void quickSort_2511531001 (int [] arr_1001,
int low_1001, int high_1001) {
        if (low_1001 < high_1001) {
            int pi_1001 = partition_1001 (arr_1001,
low_1001, high_1001);
            quickSort_2511531001(arr_1001,
low_1001, pi_1001-1);
            quickSort_2511531001(arr_1001, pi_1001+
1, high_1001);
        }
    }
}

```



```

        public static void printArr_1001 (int [] arr_1001) {
            for (int i_1001 = 0; i_1001 <
arr_1001.length; i_1001++) {
                System.out.print(arr_1001[i_1001]+ "
");
            }
            System.out.println();
        }

        public static void main (String [] args) {
            int arr_1001 [] = {10, 7, 8, 9, 1, 5};
            int n_1001 = arr_1001.length;
            System.out.print("Data sebelum diurukan: ");
            printArr_1001(arr_1001);

            quickSort_2511531001(arr_1001, 0, n_1001 -
1);
            System.out.print("Data Terurut quicksort :
");
            printArr_1001(arr_1001);
        }
    }

```

Program 2.2 : Program Algoritma Quick Sort

2.3.2 Penjelasan

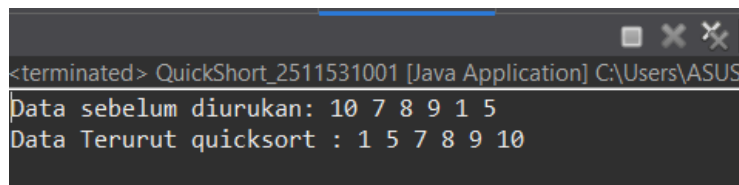
Penjelasan method method yang ada pada program ini adalah sebagai berikut :

1. Method Swap : method ini berfungsi untuk menukar elemen elemen yang ada pada array dengan menggunakan variabel tambahan yaitu temp.
2. Method medianOfthree : menentukan pivot dengan mengambil elemen paling kiri, paling kanan, dan elemen tengah dari array lalu mengatur elemen tersebut sehingga mendapatkan nilai tengah dari ketiga elemen.
3. Method partition : untuk membagi array menjadi partisi sebelum diurutkan per masing masing partisinya.
4. Method quicksort : membandingkan semua elemen yang ada dengan pivot yang telah ditentukan dan akan bernilai false jika nilai terbawah atau elemen low sudah tidak lagi lebih kecil dibandingkan dengan elemen high pada array.

5. Method printArr : berfungsi untuk menampilkan elemen yang tersimpan pada array.

Lalu pada kelas main, kita harus memasukkan elemen yang akan menjadi data yang disimpan pada array. Lalu menyimpan panjang array tersebut pada variabel n. Setelah itu program akan menggunakan method printArr untuk menampilkan angka yang belum terurut. Setelah itu, program memanggil method quicksort sebelum akhirnya akan menampilkan data yang sudah terurut.

2.3.3 Output



```
<terminated> QuickSort_2511531001 [Java Application] C:\Users\ASUS\
Data sebelum diurukan: 10 7 8 9 1 5
Data Terurut quicksort : 1 5 7 8 9 10
```

Gambar 2.2 : Output dari program algoritma Quick Sort

2.4 Program MergeSort_2511531001

2.4.1 Kode Program

```
package pekan8_25511531001;
public class MergeSort_2511531001 {
    void merge_1001 (int [] arr_1001, int l_1001, int
m_1001, int r_1001) {
        // find sizes of two subarrays to be merged
        int n1_1001 = m_1001 - l_1001 + 1;
        int n2_1001 = r_1001 - m_1001;
        /* create temp arrays */
        int L_1001 [] = new int [n1_1001];
        int R_1001 [] = new int [n2_1001];
        /*copy data to temp arrays */
        for (int i_1001 = 0; i_1001 < n1_1001;
++i_1001)
            L_1001[i_1001] =
arr_1001[l_1001+i_1001];
        for (int j_1001 = 0; j_1001< n2_1001;
++j_1001)
            R_1001[j_1001] = arr_1001 [m_1001 + 1 +
j_1001];
        int i_1001 = 0, j_1001 =0;
        //initial index merged subarray array
        int k_1001 = l_1001;
        while (i_1001 < n1_1001 && j_1001 < n2_1001)
        {
            if (L_1001[i_1001] <= R_1001[j_1001]) {
                arr_1001[k_1001] = L_1001
[i_1001];
```

```

        i_1001++;
    } else {
        arr_1001 [k_1001] = R_1001
[j_1001];

        j_1001++;
    }
    k_1001++;
}

/* copy remaining elements of L [] if any */
while (i_1001 < n1_1001) {
    arr_1001 [k_1001] = L_1001[i_1001];
    i_1001++;
    k_1001++;
}
/* copy remaining elements of R [] if any */
while (j_1001 < n2_1001) {
    arr_1001 [k_1001] = R_1001[j_1001];
    j_1001++;
    k_1001++;
}
}

void sort_1001 ( int arr_1001[], int l_1001, int
r_1001) {
    if (l_1001 < r_1001) {
        //find the middle point
        int m_1001 = (l_1001+r_1001)/2;
        //sort first and second halves
        sort_1001 (arr_1001, l_1001, m_1001);
        sort_1001 (arr_1001, m_1001 + 1 ,
r_1001);

        //merge the sorted halves
        merge_1001 (arr_1001, l_1001, m_1001,
r_1001);
    }
}

/* a utility function to print array of size n */
static void printArray_1001 (int arr_1001 []) {
    int n_1001 = arr_1001.length;
    for ( int i_1001 = 0 ; i_1001 < n_1001;
++i_1001)
        System.out.print(arr_1001 [i_1001] + "
");
    System.out.println();
}

public static void main (String args[]) {
    int arr_1001[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut: ");
    printArray_1001(arr_1001);
    MergeSort_2511531001 ob_1001 = new
MergeSort_2511531001();
    ob_1001.sort_1001(arr_1001, 0,
arr_1001.length - 1);
}

```

```

        System.out.println("\nSesudah Terurut
menggunakan merge sort");
        printArray_1001(arr_1001);
    }
}

```

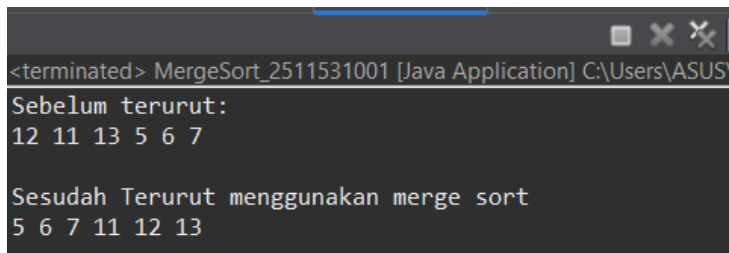
Program 2.3 : Program Algoritma Merge Sort

2.4.2 Penjelasan

Program ini dimulai dengan membuat method merge, sesuai dengan namanya, method ini akan membagi semua elemen pada array sehingga pada satu bagian hanya akan memiliki satu elemen array di dalamnya. Setelah itu ada method sort yang akan menemukan nilai tengah dari data pada array lalu akan mengurutkan bagian sebelum dan sesudah nilai tengah tersebut sebelum akan digabungkan kembali dengan method merge.

Setelah itu ada method printArray yang berfungsi untuk menampilkan elemen yang tersimpan di array. Lalu, ada kelas main yang dimulai dengan penginputan data kedalam array, lalu menampilkan array yang belum terurut, memanggil method merge sort dan setelahnya menampilkan array sudah diurutkan.

2.4.3 Output



```

<terminated> MergeSort_2511531001 [Java Application] C:\Users\ASUS
Sebelum terurut:
12 11 13 5 6 7

Sesudah Terurut menggunakan merge sort
5 6 7 11 12 13

```

Gambar 2.3 : Ouput dari program Algoritma Merge Sort

2.5 Program BubbleSortGUI_2511531001

2.5.1 Kode Program

```

package pekan8_2511531001;
import java.awt.BorderLayout;
import java.awt.Color;

import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

```

```

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.BorderFactory;
import javax.swing.JButton;

public class BubbleSortGUI_2511531001 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int [] array_1001;
    private JLabel[] labelArray_1001;
    private JButton stepButton_1001, resetButton_1001,
setButton_1001;
    private JTextField inputField_1001;
    private JPanel panelArray_1001;
    private JTextArea stepArea_1001;

    private int i_1001 = 1, j_1001;
    private boolean sorting_1001 = false;
    private int stepCount_1001 = 1;
    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511531001() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750,400);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout (new BorderLayout());

        //panel input
        JPanel inputPanel_1001 = new JPanel (new
FlowLayout());
        inputField_1001 = new JTextField (30);
        setButton_1001 = new JButton ("Set Array");
        inputPanel_1001.add(new JLabel ("Masukkan
angka (pisahkan dengan koma):"));
        inputPanel_1001.add(inputField_1001);
        inputPanel_1001.add(setButton_1001);

        //panel array visual
        panelArray_1001 = new JPanel();
        panelArray_1001.setLayout (new FlowLayout());

        //panel kontrol
        JPanel controlPanel_1001 = new JPanel ();
        stepButton_1001 = new JButton ("Langkah
Selanjutnya");
        resetButton_1001 = new JButton ("Reset");
        stepButton_1001.setEnabled(false);
        controlPanel_1001.add(stepButton_1001);

```

```

        controlPanel_1001.add(resetButton_1001);

        //Area teks untuk log langkah-langkah
        stepArea_1001 = new JTextArea (8,60);
        stepArea_1001.setEditable(false);
        stepArea_1001.setFont(new Font ("Monospaced",
Font.PLAIN,14));
        JScrollPane scrollPane_1001 = new JScrollPane
(stepArea_1001);
        //Tambahkan panel ke frame
        add(inputPanel_1001, BorderLayout.NORTH);
        add(panelArray_1001, BorderLayout.CENTER);
        add(controlPanel_1001,BorderLayout.SOUTH);
        add(scrollPane_1001, BorderLayout.EAST);

        //event Set Array
        setButton_1001.addActionListener ( e ->
setArrayFromInput_1001());

        //event Langkah Selanjutnya
        stepButton_1001.addActionListener ( e->
performStep_1001());

        //event Reset
        resetButton_1001.addActionListener(e ->
reset_1001());
    }

    private void setArrayFromInput_1001() {
        String text_1001 =
inputField_1001.getText().trim();
        if ( text_1001.isEmpty()) return;
        String [] parts_1001 = text_1001.split(",");
        array_1001 = new int [parts_1001.length];

        try {
            for ( int k_1001 = 0; k_1001 <
parts_1001.length; k_1001++) {
                array_1001 [k_1001] =
Integer.parseInt(parts_1001[k_1001].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this,
"Masukkan hanya angka yang dipisahkan" + " dengan koma !" ,
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_1001 = 0;
        j_1001 = 0;
        stepCount_1001 =1;
        sorting_1001 = true;
        stepButton_1001.setEnabled(true);
        stepArea_1001.setText("");
        panelArray_1001.removeAll();
        labelArray_1001 = new JLabel
[array_1001.length];

```

```

        for (int k_1001 = 0; k_1001 <
array_1001.length; k_1001++) {
            labelArray_1001[k_1001] = new JLabel
(String.valueOf(array_1001[k_1001]));
            labelArray_1001[k_1001].setFont(new
Font ("Arial", Font.BOLD,24));

            labelArray_1001[k_1001].setOpaque(true);

            labelArray_1001[k_1001].setBackground(Color.WHITE);

            labelArray_1001[k_1001].setBorder(BorderFactory.crea
teLineBorder(Color.BLACK));

            labelArray_1001[k_1001].setPreferredSize(new
Dimension (50,50));

            labelArray_1001[k_1001].setHorizontalAlignment(Swing
Constants.CENTER);

            panelArray_1001.add(labelArray_1001[k_1001]);
        }
        panelArray_1001.revalidate();
        panelArray_1001.repaint();
    }

    private void performStep_1001() {
        if ( !sorting_1001 || i_1001 >=
array_1001.length - 1) {
            sorting_1001 = false;
            stepButton_1001.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
            return;
        }

        resetHighlights_1001 ();
        StringBuilder stepLog_1001 = new StringBuilder();
        labelArray_1001[j_1001].setBackground(Color.cyan);
        labelArray_1001[j_1001+1].setBackground(Color.cyan);
        if (array_1001[j_1001] > array_1001 [j_1001+1]) {
            //swap
            int temp_1001 = array_1001[j_1001];
            array_1001[j_1001] = array_1001[j_1001+1];
            array_1001[j_1001+1] = temp_1001;

            labelArray_1001[j_1001].setBackground(Color.RED);

            labelArray_1001[j_1001+1].setBackground(Color.RED);
            stepLog_1001.append("Langkah
").append(stepCount_1001).append(":Menukar elemen ke-
").append(j_1001).append(" ").append(array_1001[j_1001+1])
.append(") dengan ke-
").append(j_1001+1).append("
").append(array_1001[j_1001]).append(")\n");
        } else {

```

```

        stepLog_1001.append("Langkah
").append(stepCount_1001).append(":Tidak ada pertukaran
elemen ke-").append(j_1001).append(" dan ke-
").append(j_1001+1)
        .append("\n");}
        stepLog_1001.append("Hasil :
").append(arrayToString_1001(array_1001)).append("\n\n");

        stepArea_1001.append(stepLog_1001.toString());
        updateLabels_1001();
        j_1001++;
        if (j_1001 >= array_1001.length - i_1001 - 1)
{
            j_1001 =0;
            i_1001++;
        }
        stepCount_1001++;
        if (i_1001 >= array_1001.length - 1) {
            sorting_1001 = false;
            stepButton_1001.setEnabled(false);
            JOptionPane.showMessageDialog(this,
"Sorting selsai!");
        }
    }

    private void updateLabels_1001() {
        for(int k_1001 = 0; k_1001<array_1001.length;
k_1001++) {

            labelArray_1001[k_1001].setText(String.valueOf(array
_1001[k_1001]));
        }
    }

    private void resetHighlights_1001 () {
        for (JLabel label_1001 : labelArray_1001) {
            label_1001.setBackground(Color.white);
        }
    }

    private void reset_1001() {
        inputField_1001.setText("");
        panelArray_1001.removeAll();
        panelArray_1001.revalidate();
        panelArray_1001.repaint();
        stepArea_1001.setText("");
        stepButton_1001.setEnabled(false);
        sorting_1001 = false;
        i_1001 = 0;
        j_1001 = 0;
        stepCount_1001 = 1;
    }

    private String arrayToString_1001 (int [] arr_1001)
{
        StringBuilder sb_1001 = new StringBuilder();
        for (int k_1001 = 0; k_1001 <
arr_1001.length; k_1001 ++ ) {

```



```

        sb_1001.append(arr_1001[k_1001]);
        if (k_1001 < arr_1001.length-1)
sb_1001.append(", ");
    }
    return sb_1001.toString();
}

public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                BubbleSortGUI_2511531001
frame = new BubbleSortGUI_2511531001();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}
}
}

```

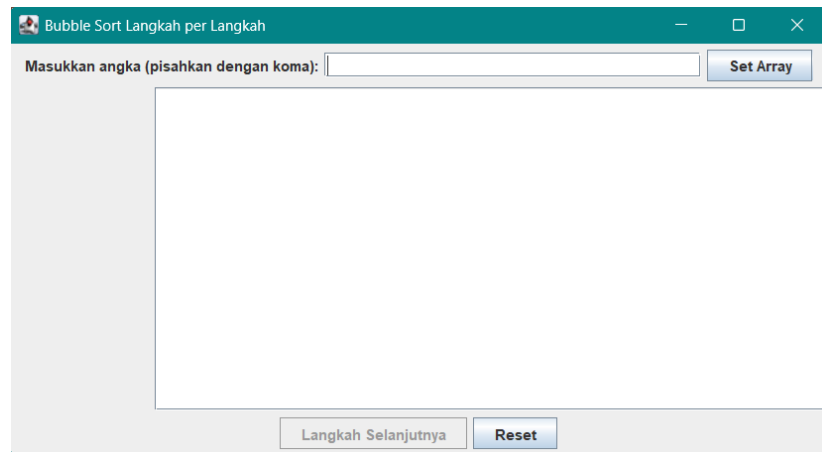
Program 2.4 : Program GUI dengan Algoritma Bubble Sort

2.5.2 Penjelasan

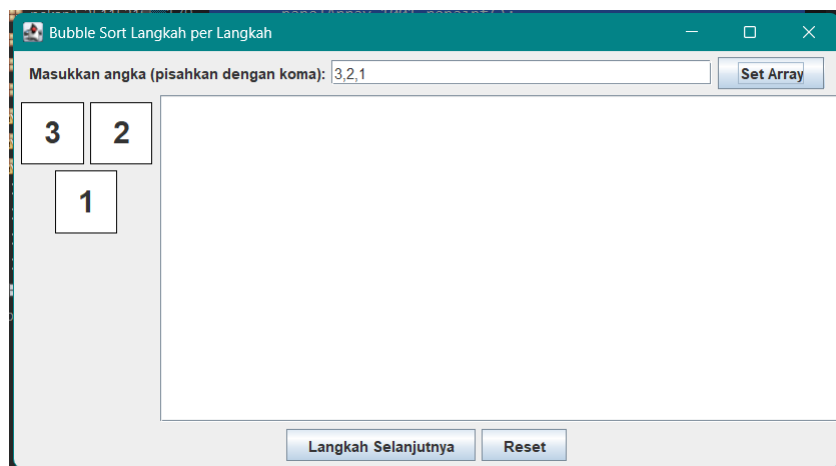
1. Rincian Objek GUI yang digunakan
 - a. JFrame : mengatur properti dasar.
 - b. JPanel : sebagai wadah untuk mengelompokkan komponen GUI. Dalam program ini variabel yang menggunakan JPanel adalah panelArray, inputPanel dan juga controlPanel.
 - c. inputField_1001 : untuk memasukkan deret angka.
 - d. setButton_1001 : memicu metode.
 - e. setArrayFromInput_1001() : untuk memproses input.
 - f. stepButton_1001 : memicu method performStep untuk menjalankan satu iterasi sorting.
 - g. resetButton_1001 : memicu method reset_1001 untuk mengembalikan aplikasi ke keadaan awal.
 - h. JTextArea: menampilkan log/catatan langkah algoritma secara detail.
 - i. labelArray_1001 : memvisualisasikan setiap elemen integer menjadi kotak visual di layar.

- j. BorderLayout : mengatur tata letak utama frame ke posisi yang diinginkan.
 - k. FlowLayout : mengatur tata letak di dalam panel- panel agar komponen tersusun rata tengah atau berurutan secara horizontal.
2. Rincian Variabel dan Method yang digunakan
- a. Variabel
 - Array_1001 : menyimpan data integer asli yang akan diproses.
 - labelArray_1001 : referensi ke komponen visual untuk update tampilan tanpa membuat ulang objek.
 - i_1001, j_1001 : menyimpan posisi indeks algoritma insertion Sort agar proses bisa dihentikan dan dilanjutkan.
 - sorting_1001 menandai apakah proses sorting sedang aktif atau sudah selesai.
 - stepCount_1001 : penghitung nomor langkah untuk ditampilkan di log.
 - b. Constructor (InsertionGUI_1001()) : mengambil string dari inputField_1001 dan memecahnya dengan split (“,”) dan mengonversinya menjadi array integer.
 - c. performStep_1001 : menjalankan satu siklus logika
 - d. updateLabels_1001 : sinkronisasi data dan tampilan.
 - e. reset_1001 : membersihkan seluruh state aplikasi.
 - f. arrayToString_1001 : konversi data.

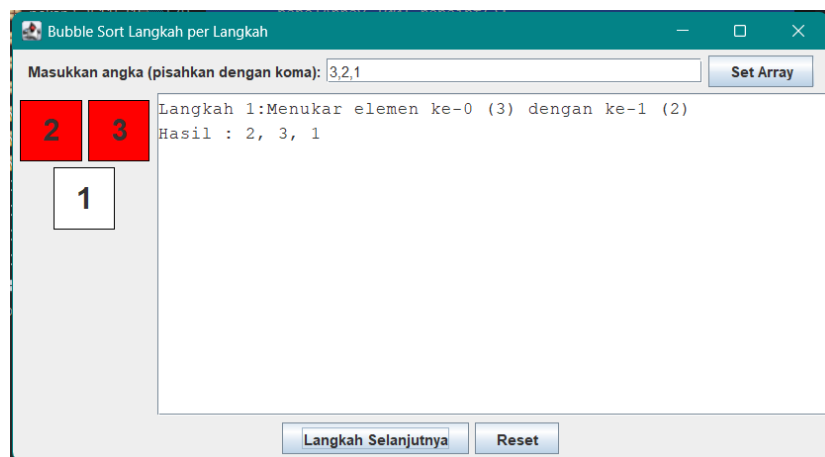
2.5.3 Output



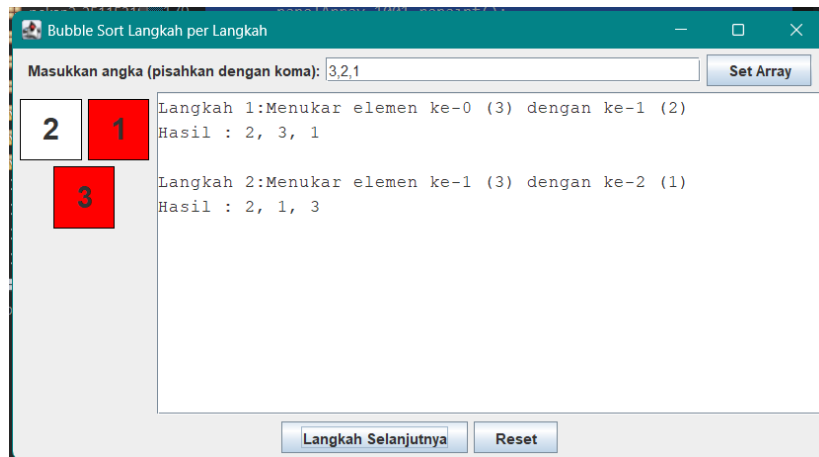
Gambar 2.4 : Tampilan awal dari GUI Algoritma Bubble Sort



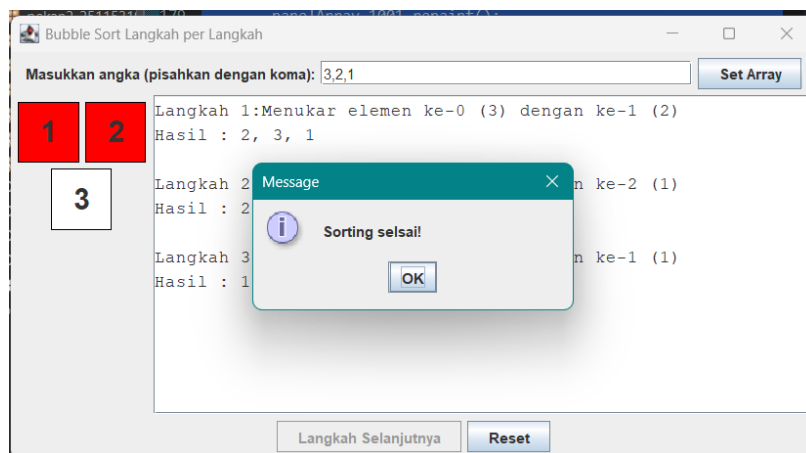
Gambar 2.5 : Menginputkan nilai pada array



Gambar 2.6 : Langkah 1 dalam proses pengurutan



Gambar 2.7 : Langkah kedua dalam proses pengurutan



Gambar 2.8 : Proses pengurutan selesai pada langkah ke 3

BAB III

KESIMPULAN

Dari praktikum yang telah dilaksanakan poin yang dapat disimpulkan adalah sebagai berikut :

1. Merge sort akan selalu membagi seluruh array menjadi dua bagian secara seimbang dan hanya memerlukan waktu linear untuk melakukan penggabungan. Kompleksitas waktu dari algoritma ini tidak terpengaruh oleh pola data awal, bahkan jika data awal bersifat kebalikan dari urutan yang diinginkan, kompleksitas waktu dari algoritma ini tetap bernilai $O(n \log n)$.
2. Pada scenario paling buruk (worst case) dari algoritma Quick Sort akan menghasilkan kompleksitas waktu yang sangat tinggi $O(n^2)$. Hal ini bisa terjadi jika pemilihan pivot terus menerus menghasilkan partisi yang timpang (misalnya pivot selalu elemen terakhir atau yang terbesar pada data yang sudah terurut).
3. Kompleksitas waktu dari algoritma Shell sangat bergantung kepada gap sequence yang digunakan. Worst case dari algoritma ini adalah $O(n^2)$. Algoritma ini sedikit menggunakan konsep algoritma insertion sort yaitu menyisipkan elemen ke urutan yang seharusnya.

DAFTAR PUSTAKA

[1] A. Fariza and Y. Setiowati, "09 Merge Sort dan Quick Sort v1," *Algoritma dan Struktur Data*, Politeknik Elektronika Negeri Surabaya, Surabaya. [Daring].

Tersedia:

https://tita.lecturer.pens.ac.id/ASD/M12_Merge+Quick/09%20Merge%20Sort%20dan%20Quick%20Sort%20v1.pdf. [Diakses: 1-Jun-2026].

[2] Yuliana, "Praktikum 14 - Sorting Merge Sort," *Struktur Data*, Politeknik Elektronika Negeri Surabaya, Surabaya, 2015. [Daring]. Tersedia:

<https://yuliana.lecturer.pens.ac.id/Struktur%20Data/PRAKTIKUM%202015/Praktikum%2014%20-%20Sorting%20Merge%20Sort.pdf>. [Diakses: 1-Jun-2026].

[3] U. S. Tita Karlita, E. Martiana, K. Arna Fariza, "Shell Sort," *Algoritma dan Struktur Data*, Politeknik Elektronika Negeri Surabaya, Surabaya, 2021. [Daring].

Tersedia:

[https://tita.lecturer.pens.ac.id/ASD2021/M11%20-%20Sorting%202%20\(bubble%20&%20shell\)/shell-sort.pdf](https://tita.lecturer.pens.ac.id/ASD2021/M11%20-%20Sorting%202%20(bubble%20&%20shell)/shell-sort.pdf). [Diakses: 1-Jun-2026].

LAPORAN TUGAS

A. Kode Program

```
package pekan8_2511531001;

class Lagu_2511531001 {
    //deklarasi variabel
    String judul_1001;
    String penyanyi_1001;
    int durasi_1001;

    // konstruktor
    public Lagu_2511531001 (String j_1001, String p_1001, int
d_1001 ) {
        this.judul_1001 = j_1001;
        this.penyanyi_1001 = p_1001;
        this.durasi_1001 = d_1001;
    }
}

public class Sorting_2511531001 {

    Lagu_2511531001 [] dataLagu_1001;
    int jumlahLagu_1001;

    public Sorting_2511531001 () {
        dataLagu_1001 = new Lagu_2511531001 [20]; //array maksimal
hanya berisikan 20 data lagu
        jumlahLagu_1001 = 0;
    }

    // method mengisi data awal (minimal 7 lagu)
    public void inputData_1001 () {
        dataLagu_1001 [0] = new Lagu_2511531001("Just The Way You
Are", "Bruno Mars", 221);
        dataLagu_1001 [1] = new Lagu_2511531001("Grenade", "Bruno
Mars", 222);
        dataLagu_1001 [2] = new Lagu_2511531001("Locked Out Of
Heaven", "Bruno Mars", 233);
        dataLagu_1001 [3] = new Lagu_2511531001("When I Was Your
Man", "Bruno Mars", 214);
        dataLagu_1001 [4] = new Lagu_2511531001("Treasure", "Bruno
Mars", 178);
        dataLagu_1001 [5] = new Lagu_2511531001("The Lazy Song",
"Bruno Mars", 190);
        dataLagu_1001 [6] = new Lagu_2511531001("That's What I Like",
"Bruno Mars", 208);
        dataLagu_1001 [7] = new Lagu_2511531001("Versace on the
Floor", "Bruno Mars", 261);
        dataLagu_1001 [8] = new Lagu_2511531001("Marry You", "Bruno
Mars", 230);
        jumlahLagu_1001 = 9;
    }

    //method menampilkan data sebelum/sesudah sorting
    public void tampilData_1001() {
```

```

        for (int i_1001 = 0; i_1001 < jumlahLagu_1001; i_1001++) {
            System.out.println((i_1001+1) + ". " +
dataLagu_1001[i_1001].judul_1001 + "- " +
dataLagu_1001[i_1001].durasi_1001 + " detik");
        }
    }

    //method utama quick sort (urutan durasi ascending)
    public void quickSort_1001 () {
        quickSort_1001 (dataLagu_1001, 0, jumlahLagu_1001 -1);
    }

    // rekursif quick sort
    private void quickSort_1001 (Lagu_2511531001 [] arr_1001, int
low_1001, int high_1001) {
        if (low_1001 < high_1001) {
            int pi_1001 = partition_1001 (arr_1001, low_1001,
high_1001);

            quickSort_1001(arr_1001, low_1001, pi_1001-1);
            quickSort_1001(arr_1001, pi_1001+ 1, high_1001);
        }
    }

    private int partition_1001 (Lagu_2511531001 [] arr_1001, int
low_1001, int high_1001) {
        int pivot_1001 = arr_1001[high_1001].durasi_1001;
        int i_1001 = low_1001 - 1;
        for (int j_1001 = low_1001; j_1001 < high_1001 ; j_1001++)
        {
            if (arr_1001[j_1001].durasi_1001 <= pivot_1001) {
                i_1001++;

                Lagu_2511531001 temp_1001 = arr_1001
[i_1001];
                arr_1001 [i_1001] = arr_1001 [j_1001];
                arr_1001 [j_1001] = temp_1001;
            }
        }
        Lagu_2511531001 temp_1001 = arr_1001[i_1001 + 1];
        arr_1001 [i_1001+1] = arr_1001[high_1001];
        arr_1001 [high_1001] = temp_1001;
        return i_1001 + 1;
    }

    public static void main (String [] args) {
        Sorting_2511531001 playlist_1001 = new
Sorting_2511531001();

        System.out.println("=== Sorting Playlist dengan Algoritma
Quick Sort ===");
        System.out.print("NIM = 2511531001");

        playlist_1001.inputData_1001();
        System.out.println("\nData sebelum Sorting: ");
        playlist_1001.tampilData_1001();

        playlist_1001.quickSort_1001();
    }

```



```

        System.out.println("\n Data Setelah Quick Sort (Durasi
Asc): ");
        playlist_1001.tampilData_1001();
    }
}

```

B. Output

```

=== Sorting Playlist dengan Algoritma Quick Sort ===
NIM = 2511531001
Data sebelum Sorting:
1. Just The Way You Are- 221 detik
2. Grenade- 222 detik
3. Locked Out Of Heaven- 233 detik
4. When I Was Your Man- 214 detik
5. Treasure- 178 detik
6. The Lazy Song- 190 detik
7. That's What I Like- 208 detik
8. Versace on the Floor- 261 detik
9. Marry You- 230 detik

Data Setelah Quick Sort (Durasi Asc):
1. Treasure- 178 detik
2. The Lazy Song- 190 detik
3. That's What I Like- 208 detik
4. When I Was Your Man- 214 detik
5. Just The Way You Are- 221 detik
6. Grenade- 222 detik
7. Marry You- 230 detik
8. Locked Out Of Heaven- 233 detik
9. Versace on the Floor- 261 detik

```

C. Penjelasan

Program di atas menggunakan algoritma Quick Sort dan mengurutkan durasi dari masing masing lagu yang dimasukkan ke dalam array playlist lagu. Pemilihan algoritma ini berlandaskan karena karakteristik dari algoritma quick Sort yang terhitung cukup efisien untuk mengurutkan data dalam jumlah sedang hingga besar. Algoritma ini juga lebih baik jika dibandingkan dengan merge sort dan algoritma ini memiliki kompleksitas rata rata yang cukup efisien yaitu $O(n \log n)$. Walaupun, kompleksitas waktu worst case pada algoritma ini bisa $O(n^2)$ tetapi ini bisa diminimalkan dengan strategi pengecekan data awal terlebih dahulu sehingga menghasilkan pivot yang ideal. Alasan berikutnya adalah dikarenakan penulis sendiri ingin lebih memahami konsep pivot pada algoritma quick sort dikarenakan konsep ini terhitung cukup sulit dipahami oleh penulis sendiri.